INFORMATION TECHNOLOGY

Faculty of Information Technology and Digital Innovation
King Mongkut's University of Technology North Bangkok

Chapter 3:
Dart Language Essentials

Name: Mr.Tongpool Heeptaisong (Lex)
Email: tongpool.h@itd.kmutnb.ac.th
Line id: ilexkung
Facebook: https://www.facebook.com/tongpool.heeptaisong
Mobile: 09-097-1071-9

Chapter 3:
Dart Language Essentials

Objectives

- Understand what is Dart.
- Understand Basic Dart Program.
- Coding and Debugging Dart.

Chapter 3:
Dart Language Essentials

Topics:

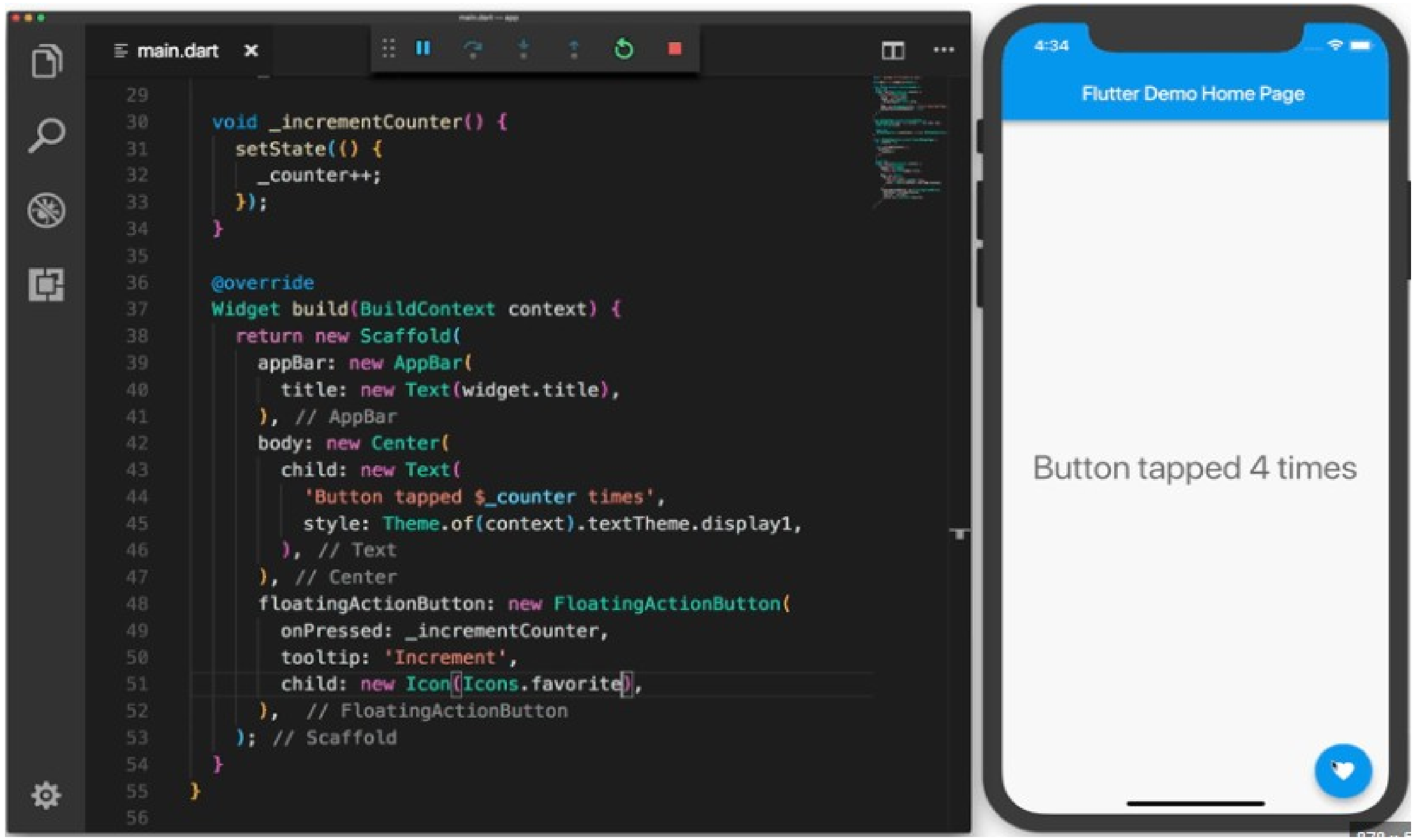
- Introduction to Dart
- Basic Dart Program
- Built-in Types in Dart: Number, Strings, Booleans, List, Set, and Maps
- Dart Variables
- Constant Variables
- Dart Functions
- Dart Operators
- Control Flow
- Dart Classes
- Dart in Flutter

Topic 1:
What is the Dart

Why do we need to learn dart in this course, and why Flutter is using dart.

Dart

- ✓ Supported by Google
  - Google used in backend projects
- ✓ Free and Open Source
  - Everyone can use and contribute
- ✓ Fast on All Platforms
  - Support mobile, web, backend



Topic 1:
What is the Dart

Dart is similar to modern world language, friendly for all developer.

test.dart

Dart > test.dart > ...

```dart
void prinInteger(int n) {
  print('The number is $n.');
}

void main() {
  var n = 42;
  prinInteger(n);
}
```

Topic 2:
Built-in Types in Dart

Built-in Types in Dart:

- Number
- Strings
- Booleans
- Lists
- Set (math set)
- Map (key-value)

Numbers

Integer

```
int x = 1;
int hex = 0xDEADBEEF; (hexadecimal)
int exp = 8e5; (exponential)
```

Double

```
double y = 1.1;
double exp = 1.42e5; (exponential)
```

Num (can be either int or double)

```
num x = 1; (initialize as integer)
num = 1.5; (can be reassigned as double)
```

Topic 2:

Built-in Types in Dart

Built-in Types in Dart:

- Number
- Strings
- Booleans
- Lists
- Set (math set)
- Map (key-value)

Strings

Works with both single/double quotes

```
String s1 = 'This uses single quote';
String s2 = "This uses double quotes";
String s3 = 'It\'s easy to escape the string delimiter.';
(escape ' character with \ when using single quote)"
String s4 = "It's even easier to use the other delimiter.";
```

String Concatenation

```
String welcome = 'Welcome ''to the'" Skooldio course";
```

Multi-line String

```
String welcome = '''
                Welcome to
                Skooldio course
                ''';
```

String Interpolation (using variables in a string)

```
String skd = "Skooldio";
"String welcome = ""Welcome to $skd"";
(Welcome to Skooldio) (Direct Usage)"
"String welcome = ""Welcome to ${skd.toUpperCase()}"";
(Welcome to SKOOLDIO) (Call method inside {})
```

Topic 2:
Built-in Types in Dart

Built-in Types in Dart:

- Number
- Strings
- Booleans
- Lists
- Set (math set)
- Map (key-value)

Booleans

```
bool b1 = true;
bool b2 = false;
int num = 42;
bool isLessThanZero = num < 0; //false
```

Lists

```
List<int> list1 = [1, 2, 3];
 List<String> list2 = [
  'Car',
  'Boat',
  'Plane', // trailing-comma
];
```

List Concatenation

```
List<int> list = [1, 2, 3];
List<int> list2 = [0, ...list];
// [0, 1, 2, 3]
```

Topic 2:
Built-in Types in Dart

Built-in Types in Dart:
- Number
- Strings
- Booleans
- Lists
- Set (math set)
- Map (key-value)

Collection if/for

```
List<String> nav = ['Home', if (condition) 'Outlet'];

// ['Home'] if condition is false
// ['Home', 'Outlet'] if condition is true
```

```
List<int> list1 = [1, 2, 3];
List<String> list2 = ['#0', for (int i in list1) '#$i'];

// ['#0', '#1', '#2', '#3']
```

List Operations

```
List<int> numbers = [1, 2, 3];

numbers.length; // 3
numbers.first; // 1
numbers.last; // 3
numbers.isEmpty; // false
numbers.isNotEmpty; // true

numbers.add(4);
print(numbers); // [1, 2, 3, 4];

List<int> square = numbers.map((number) => number * number).toList();
// [1, 4, 9, 16]

List<int> even = numbers.where((number) => number.isEven).toList();
// [2, 4]

num total = numbers.reduce((total, number) => total + number);
// 10
```

Topic 2:
Built-in Types in Dart

Built-in Types in Dart:

- Number
- Strings
- Booleans
- Lists
- Set (math set)
- Map (key-value)

Set

```
Set<String> names = {};
```

Note (When initializing as dynamic type.)

```
var names = {}; // This is a Map
var names = <String>{}; // This is a Set
```

Set Operations

```
var oldSet = <String>{};
var newSet = {'A', 'B'};

oldSet.add('C');
oldSet.addAll(newSet);
print(oldSet); // {'C', 'A', 'B'}

oldSet.remove('A');
print(oldSet); // {'C' ,'B'}
```

```
var set1 = {1, 2, 3};
var set2 = {2, 3, 4};

var unionSet = set1.union(set2);
var intersectSet = set1.intersection(set2);
var diffSet = set1.difference(set2);

print(unionSet); // {1, 2, 3, 4}
print(intersectSet); // {2, 3}
print(diffSet); // {1}
```

Topic 2:
Built-in Types in Dart

Built-in Types in Dart:

- Number
- Strings
- Booleans
- Lists
- Set (math set)
- Map (key-value)

Map

```
Map<int, String> nobleGases = Map<int, String>();
nobleGases[2] = 'helium';
nobleGases[10] = 'neon';
nobleGases[18] = 'argon';

is the same as

var nobleGases = {
   2: 'helium',
   10: 'neon',
   18: 'argon',
};
```

Map Operations

```
var map1 = {
   'first': '1',
   'second': '2',
   'fifth': '5'
};

var map2 = {
   'third': '3',
   ...map1 // concatenation
};

print(map2);
// {third: 3, first: 1, second: 2, fifth: 5}

map2.containsKey('first'); // true
map2.containsValue('first'); // false

// Loop over key-value
map2.forEach((key, value) => print('$key - $value'));
```

Topic 3:
Dart Variables

Dart Variables:

- Initialization
- Nullable (Null-safety)

- Late Variables
  - must be initialized later

- Constant Values
  - once the variable is initialized it cannot be altered

- final vs const

Variables

Initialization

```
var s1 = 'Hello'; // infer type
String s2 = 'World'; // explicit type
print('$s1 $s2');
// Hello World

s2 = 'Flutter'; // can be reassigned
print('$s1 $s2');
// Hello Flutter
```

Nullable

```
String? name;
int? age;
```

Topic 3:
Dart Variables

Dart Variables:

- Initialization
- Nullable (Null-safety)

- Late Variables
  - must be initialized later

- Constant Values
  - once the variable is initialized it cannot be altered

- final vs const

Late Variables

must be initialized later

```
late String name;
late int age;

name = 'A';

print(name); // A
print(age); // Error 'age' is unassigned
```

Constant Values

once the variable is initialized it cannot be altered

```
final name = 'Hello';
name = 'World';

// Error: The final variable 'name' can only be set once.

const name = 'Hello';
name = 'World';
// Error: Constant variables can't be assigned a value.
```

Topic 3:
Dart Variables

Dart Variables:

- Initialization
- Nullable (Null-safety)
- Late Variables
  - must be initialized later
- Constant Values
  - once the variable is initialized it cannot be altered
- final vs const

final vs const

const -> value assigned at compile time.
final -> value assigned at runtime.

```
final time1 = DateTime.now();
print(time1); //2022-01-01 00:00:00.001

const time2 = DateTime.now(); // Error: The constructor being
// called isn't a const constructor.
```

Topic 4:
Dart Functions

Dart Functions:

- main() function
- Function Creation
- Anonymous Function
- Function as a parameter

Functions

main() function

```
void main() {
  print('Hello, Skooldio!');
}
```

Function Creation

```
// type-annotated
int multiply(int num1, int num2) {
  return num1 * num2;
}

// type-omitted
int multiply(num1, num2) {
  return num1 * num2;
}

// shorthand syntax
int multiply(num1, num2) => num1 * num2;
```

Topic 4:
Dart Functions

Dart Functions:

- main() function
- Function Creation
- Anonymous Function
- Function as a parameter

Anonymous Function

```
const fruits = ['apples', 'bananas', 'oranges'];

// print each fruit
fruits.forEach((fruit) {
  print(fruit);
});

// apples
// bananas
// oranges
```

Function as a parameter

```
void printItem(item) => print(item);

const fruits = ['apples', 'bananas', 'oranges'];
fruits.forEach(printItem);

// apples
// bananas
// oranges
```

Topic 5:
Dart Operators

Dart Operators:

- Unary Operators
- Arithmetic Operators
- Logical Operators
- Conditional Operators

Operators

Unary Operators

Postfix

```
i++;
i--;
// expr++, expr--

helloWorld()
// ()

someList[0] nullableList ?[0]
// [] ?[]

someObject. nullableObject?.
// . ?.
```

Prefix

```
-expr       !expr       ~expr

++expr      --expr      await expr
```

Arithmetic Operators

```
+ (plus)       - (minus)       * (multiply)

/ (divide)     % (modulo)      ~/ (integer division)
```

Logical Operators

```
A == B
A != B
A && B
A || B
```

Conditional Operators

```
thisIsTrue ? thenThis : orThat

int? nullableObject;
print(nullableObject ?? 0)
// 0
```

Topic 6:
Dart Control Flow

Dart Control Flow:

- if - else
- Loops

Control Flow

if - else

```
if (isRaining()) {
   you.bringRainCoat();
} else if (isSnowing()) {
   you.wearJacket();
} else {
   car.putTopDown();
}
```

Loops

For Loop

```
int sum = 0;

for (int i = 1; i <= 5; i++){
   sum += i;
}

print(sum); // 6
```

```
List<int> numbers = [1, 2, 3];

for(var number in numbers) {
   print(number);
}

numbers.forEach((number) => print(number));
```

Topic 6:
Dart Control Flow

Dart Control Flow:

- if - else
- Loops

While Loop

```
while (!isDone()) {
  doSomething();
}
```

Do While Loop

```
int i = 1;
String s1 = '';
do {
  s1 += i.toString();
  i++;
} while (i < 5);

print(s1); // 1234

int j = 1;
String s2 = '';
do {
  s2 += j.toString();
  // stop at 5
  if(j == 5) {
    break;
  }
  j++;
} while (j < 10);
print(s2); // 12345
```

Topic 7:
Dart Classes

Dart Classes:

- OOP concept in dart world
- Data
- Function
- Inheritance

Classes

```
class Point {
  double x = 0;
  double y = 0;

  // Constructor
  Point(double x, double y) {
    this.x = x;
    this.y = y;
  }
  // or
  // Point({required this.x, required this.y});

  // Named Constructors
  Point.origin() {
    this.x = 0;
    this.y = 0;
  }

  Point.onX(this.y) : x = 0;

  // Getters
  double get getX => x;
  int get yAsInt => y.toInt();
  double get distanceFromOrigin => sqrt(x * x + y * y);

  // Setters
  set setX(double x) => this.x = x;
  set setY(double y) => this.y = y;

  // Methods
  void multiplyX(double x) {
    this.x *= x;
  }

  String toString() {
    return 'Point($x, $y)';
  }
}
```

Topic 7:
Dart Classes

Dart Classes:

- OOP concept in dart world
- Data
- Function
- Inheritance

Inheritance

```
class Animal {
  String name;
  int age;

  Animal(this.name, this.age);

  void eat() {
    print('Animal is eating');
  }
}

class Dog extends Animal {
  String breed;

  Dog(String name, int age, String breed) : super(name, age) {
    this.breed = breed;
  };

  get breed => breed;

  // override existing method
  @override
  void eat() {
    print('Dog is eating');
  }
}
```

030513908-65

MOBILE APPLICATION DEVELOPMENT

Teaching and Learning Resources

Flutter Developer

YouTube

- The Flutter Way — 219K subscribers
- Mitch Koko — 82.8K subscribers
- Flutter Mapp — 181K subscribers
- Learn App Code — 18.8K subscribers
- dbestech — 135K subscribers
- WTF Code — 13.1K subscribers

- HeyFlutter.c... — 110K subscribers
- UiLover — 59.6K subscribers
- Coding With T — 124K subscribers
- Dear Programmer — 32.4K subscribers
- Shivam Gupta — 14.4K subscribers
- CodentraTech — 158 subscribers

- Nylo — 656 subscribers
- Flutter Hero — 1.19K subscribers
- Flutter — 621K subscribers
- Tech With Otabek — 4.55K subscribers
- Yaqoob Developer — 62.3K subscribers
- ART PROGRAMS — 7.19K subscribers

UXUI Design

- Afgprogrammer — 32.6K subscribers
- Optimizers — 4.26K subscribers
- Figma — 464K subscribers

Chapter 3:
Dart Language Essentials

References:

- Introduction to Dart. at https://dart.dev/language
- Dart Programming Tutorial. at https://www.tutorialspoint.com/dart_programming/index.htm
- Dart Tutorial. at https://dart-tutorial.com/
- Dart Code. at https://dartcode.org/